

Normalization and Pre-tokenization

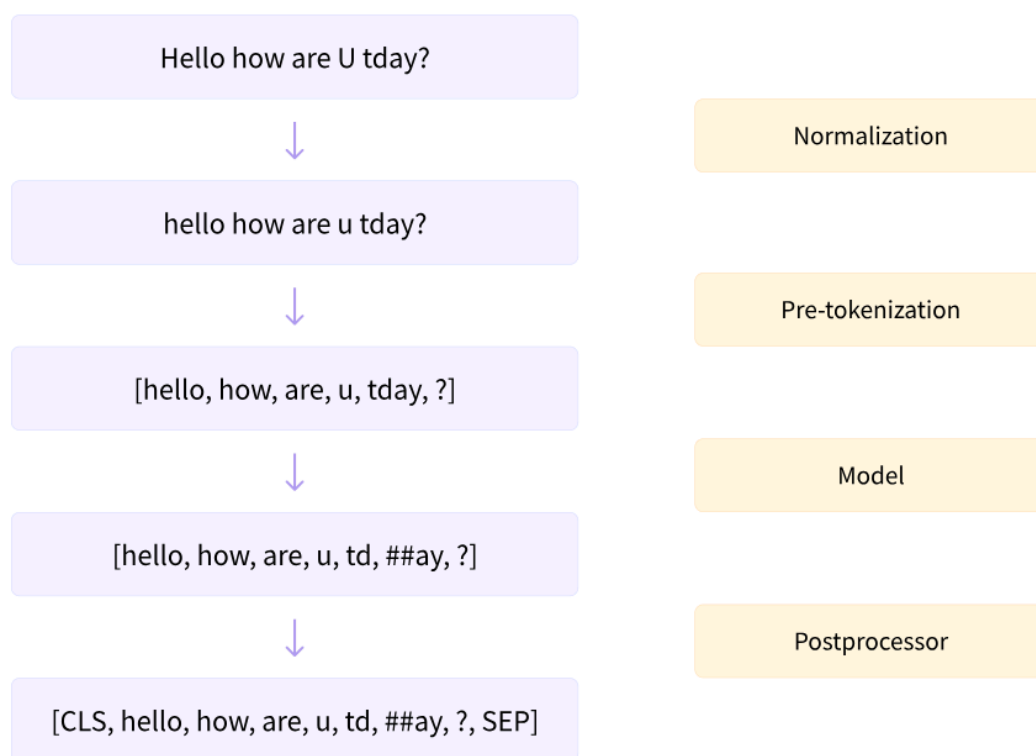
The three most common subword tokenization algorithms used in Transformer models are:

- Byte-Pair Encoding (BPE)
- WordPiece
- Unigram

Before understanding these algorithms in detail, it is first necessary to know what steps a tokenizer follows at the beginning when processing text.

In general, the steps of the tokenization pipeline are:

Text -> Normalization -> Pre-tokenization -> Subtokenization



That is, before splitting a text into small subtokens according to the model, the tokenizer usually performs two important tasks:

- Normalization

- Pre-tokenization

1. What is Normalization?

Normalization is the process of bringing text into a clean and standard form.

In this step, the following tasks are usually done:

- Removing unnecessary whitespace
- Lowercasing all letters
- Removing accent or diacritic marks
- Performing Unicode normalization, such as NFC or NFKC

Simply put, the purpose of normalization is to make the text uniform in such a way that the tokenizer can later process it easily and consistently.

2. The backend tokenizer of the Transformers tokenizer

Inside the tokenizer of the Transformers library, there is an attribute called `backend_tokenizer`. Through this, access can be obtained to the underlying tokenizer of the Tokenizers library.

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")  
print(type(tokenizer.backend_tokenizer))
```

Output:

```
<class 'tokenizers.Tokenizer'>
```

This means that inside the Transformers tokenizer, `tokenizers.Tokenizer` is actually working.

3. Seeing how Normalization works

Inside the `normalizer` attribute of the tokenizer, there is a method called `normalize_str()`. By using this, it can be understood what changes occur when normalization is applied to a string.

```
print(tokenizer.backend_tokenizer.normalizer.normalize_str("Héllò hõw are ü?"))
```

Output:

'hello how are u?'

4. What changed here?

Since we used the `bert-base-uncased` tokenizer, it made the following changes:

- Lowercased all letters
- `Héllò` became `hello`
- Removed accents
- `é, ò, ô, ü` changed to `e, o, o, u`

So the whole string:

`Héllò hôw are ü?`

changed into:

`hello how are u?`

5. What is Pre-tokenization?

After normalization comes pre-tokenization.

In this step, the text is initially split in such a way that the tokenizer can later create subtokens easily.

Pre-tokenization can usually do the following tasks:

- Separating text according to spaces
- Separating punctuation
- Identifying word boundaries

For example:

`Hello, world!`

Through pre-tokenization, this could be split like this:

- `Hello`

- ,
- world
- !

That is, even before final subtokenization begins, the text is split into small meaningful parts.

Pre-tokenization

As we will see in the later sections, a tokenizer cannot be trained only on raw text. Instead, first we need to split the texts into small entities, such as words. This is where the pre-tokenization step comes in. As we saw in Chapter 2, a word-based tokenizer can easily split raw text into words based on whitespace and punctuation. Those words will be the boundaries of the subtokens that the tokenizer can learn during its training.

To see how a fast tokenizer performs pre-tokenization, we can use the `pre_tokenize_str()` method of the `pre_tokenizer` attribute of the tokenizer object:

```
tokenizer.backend_tokenizer.pre_tokenizer.pre_tokenize_str("Hello, how are you?")

[('Hello', (0, 5)), (',', (5, 6)), ('how', (7, 10)), ('are', (11, 14)), ('you', (16, 19)), ('?', (19, 20))]
```

Notice that the tokenizer is already tracking the offsets, and this is how it can give us the offset mapping, which we used in the previous section. Here the tokenizer ignores the two spaces and replaces them with only one, but a jump is seen in the offset between `are` and `you` so that it can account for that.

Since we are using a BERT tokenizer, pre-tokenization here involves splitting based on whitespace and punctuation. The rules for this step can be different for other tokenizers. For example, if we use the GPT-2 tokenizer:

```
tokenizer = AutoTokenizer.from_pretrained("gpt2")
tokenizer.backend_tokenizer.pre_tokenizer.pre_tokenize_str("Hello, how are you?")
```

This will also split on whitespace and punctuation, but it will keep the spaces and replace them with the `Ġ` symbol, so that when the tokens are decoded, the original spaces can be recovered again:

```
[('Hello', (0, 5)), (',', (5, 6)), ('Ġhow', (6, 10)), ('Ġare', (10, 14)), ('Ġ', (14, 15)), ('Ġyou', (15, 19)), ('?', (19, 20))]
```

Also note that unlike the BERT tokenizer, this tokenizer does not ignore the double space.

As one final example, let us look at the T5 tokenizer, which is based on the SentencePiece algorithm:

```
tokenizer = AutoTokenizer.from_pretrained("t5-small")
tokenizer.backend_tokenizer.pre_tokenizer.pre_tokenize_str("Hello, how are you?")
```

```
[['_Hello,', (0, 6)], ['_how', (7, 10)], ['_are', (11, 14)], ['_you?', (16, 20)]]
```

Like the GPT-2 tokenizer, this also keeps spaces and replaces them with a specific token (`_`), but the T5 tokenizer splits only on whitespace, not on punctuation. Also note that by default it added a space at the beginning of the sentence (before `Hello`) and ignored the double space between `are` and `you`.

Now that we have seen how some different tokenizers process text, we can now begin exploring the underlying algorithms themselves. We will start with a brief introduction to the widely applicable SentencePiece; then, in the next three sections, we will examine how the three main algorithms used in subword tokenization work.

SentencePiece

SentencePiece is a tokenization algorithm for text preprocessing, which you can use with any of the models that we will see in the next three sections. It considers text as a sequence of Unicode characters, and replaces spaces with a special character `_`. When it is used together with the Unigram algorithm (see section 7), it does not even require a pre-tokenization step, which is very useful for languages where the space character is not used (such as Chinese or Japanese).

Another main feature of SentencePiece is reversible tokenization: since there is no separate special treatment for spaces, token decoding is done simply by concatenating them and replacing `_` with spaces — the result is normalized text. As we saw earlier, the BERT tokenizer removes repeated spaces, so its tokenization is not reversible.

Algorithm overview

In the following sections, we will dive deeper into the three main subword tokenization algorithms: BPE (used by GPT-2 and others), WordPiece (used, for example, in BERT), and Unigram (used by T5 and others). Before getting started, below is a brief overview of how each

of them works. If the subject is not yet understandable to you, do not hesitate to come back to this table after reading the next sections.

Model	BPE	WordPiece	Unigram
Training	Starts from a small vocabulary and learns rules for merging tokens	Starts from a small vocabulary and learns rules for merging tokens	Starts from a large vocabulary and learns rules for removing tokens
Training step	Merges the tokens related to the most common pair	Merges the tokens related to the pair with the best score based on the frequency of the pair, and gives priority to pairs where each individual token is less frequent	Removes all the tokens in the vocabulary that will minimize the loss calculated over the whole corpus
Learns	Merge rules and a vocabulary	Only a vocabulary	A vocabulary with a score for each token
Encoding	Splits a word into characters and applies the merges learned during training	Finds the largest subword from the beginning that is in the vocabulary, then does the same for the rest of the word	Finds the most likely split into tokens using the scores learned during training

Now let us move into BPE.

Need this as a clean note version too হলে বলো.